Core Functionality

Scope and development record

This Week 2 milestone implements a reservation subset of the Campus Equipment Checkout software project: FR-02, FR-04, FR-05 and FR-06. It extends the existing Django inventory application rather than the superseded research-input generator. Work and assistant-run verification were performed on October 27, 2026. The milestone title is not a claim that development occurred in an earlier calendar week.

Selected requirements and observable behavior

FR-02: Server-side authentication, reviewer permission and borrower ownership are enforced for implemented routes. Staff status alone does not permit approvals. Borrowers see only their own requests; posted borrower/status fields cannot override the server-selected owner or Pending status. Full-system role coverage remains partial until lending features exist.

FR-04: Authenticated users search active equipment by name or category and enter a future start/end interval. The catalog identifies overlap with approved reservations. Pending requests do not block availability; intervals touching only at an endpoint do not overlap.

FR-05: A valid future request for active equipment creates a persistent Pending reservation linked to the logged-in borrower. Reversed/equal times, past starts, malformed dates, and inactive equipment are rejected. Time controls use America/New_York; timestamps are timezone aware.

FR-06: A user with inventory.review_reservation can approve or reject a pending request. Approval rechecks active equipment, future time, and conflicting approved reservations. Failed approval leaves the request Pending, without reviewer metadata. Repeat decisions are rejected.

Core Functionality / continued

Source modules and component interaction

inventory/models.py and migrations/0002_reservation.py define Reservation, protected user/equipment foreign keys, review metadata, a positive-interval database constraint, and an overlap-query index. Existing inventory records remain intact when the migration is applied.

inventory/forms.py validates browser times. inventory/services.py owns request creation and decision rules. inventory/views.py connects authenticated routes, forms, services and rendered responses; config/urls.py maps them. inventory/templates provides login, catalog, request form, borrower history, and reviewer queue.

POST /equipment/<id>/reserve/ creates requests. GET /reservations/ displays only the caller's records. GET /review/ and POST /review/<id>/ require reviewer permission. GET requests cannot approve or reject; POSTs use Django CSRF middleware. These are HTML routes, not a separate JSON API.

Transaction design and technical problems

SQLite does not implement SELECT FOR UPDATE locking. The application uses short transaction.atomic service calls with SQLite transaction_mode=IMMEDIATE, so writer serialization begins before the conflict read. This avoids a separate check-then-write race. The design follows the documented SQLite transaction option (Django Software Foundation n.d.).

Concurrent tests use a file-backed database and independent connections. An in-memory test database would not represent the intended file-lock behavior. A 20-second busy timeout is configured; the UI displays a retry message if database contention raises OperationalError. This is not a scalability guarantee.

Reservation is deliberately not registered as an editable Django admin model: ordinary UI approvals must pass through the service checks. Direct SQL or privileged Python-shell writes can bypass business rules and are outside the supported application interface.

Core Functionality / continued

Automated tests and measured results

Command: python manage.py test inventory.test_reservations.ReservationTests --noinput --verbosity 2. Result: 14 tests passed. Actual verbose output is retained in week_2/verification/test_output.txt.

Coverage includes persistence of Pending and owner; invalid intervals; inactive equipment; database interval constraint; half-open boundaries; conflict rollback; different equipment; rejection and repeated decisions; retirement/time rechecks; borrower and unprivileged-staff denials; forged owner/status; login/logout; search and bad input; CSRF and GET denials; and the integrated request/approval workflow.

The full Week 3 suite adds six original inventory tests and a concurrent test running ten two-connection rounds. All 21 tests passed in 2.127 seconds in the recorded run. Each concurrency round produced one approval and one conflict. This finite test supports the local prototype behavior, not proof for all deployments.

Incomplete work and changes

The Django/SQLite architecture is retained. New borrower-facing templates and a service layer connect the original inventory component to reservations. Root URL now opens the catalog rather than redirecting to admin; /admin/ remains available. No new dependencies were required.

Cancellation, checkout, return, overdue detection and usage reporting remain planned. Login is implemented for administrator-created accounts; registration and university SSO remain out of scope. No NFR performance benchmark, Safari compatibility run, or full accessibility audit has been completed.

Reference and assistance

Django Software Foundation. n.d. “Databases.” Django Documentation. Accessed October 27, 2026. https://docs.djangoproject.com/en/5.1/ref/databases/.

Implementation, test execution and report preparation used OpenAI assistant support. The student subsequently ran the browser workflow with guidance and all 21 tests passed in 1.791 seconds, as documented in Week 3 student_run screenshots. Review of the code and explanation of its rules remain the student’s responsibility.